

RTL-Level Specification of Camouflaging Intent for Integrated Circuit IP Protection

Anonymous

Abstract—Camouflaging technology protects integrated circuit (IC) intellectual property (IP) against reverse-engineering by manufacturing a subset of gates whose Boolean functions cannot be determined through physical inspection alone. Existing design flows apply camouflaging at the gate level after synthesis, forcing the designer to select individual gates without a principled correspondence to the high-level algorithmic secrets she wishes to protect. We propose a lightweight HDL annotation—the `camouflage` construct—that lets a designer declare trade-secret logic directly in VHDL or Verilog source. A synthesis tool supporting the construct compiles each marked region into a *universal circuit*: a fixed, publicly visible logic structure that can realise any Boolean function of a given arity, configured entirely by a set of constant bits that are embedded as camouflaged cells during fabrication. Because the universal circuit topology is function-agnostic by construction, no designer-specified decoy is required: an attacker who recovers the topology learns only an upper bound on the complexity of the protected function, not the function itself. We define the construct’s syntax and semantics, describe the corresponding synthesis obligations, and analyse the approach on the thread-scheduler and memory-arbiter of the OpenSPARC T1 processor. Using the best known universal circuit construction, the configuration bits that encode the protected functions amount to $O(s \log s)$ camouflaged cells for a protected region of s gates. These cells hide individual bits rather than gate functions, and are therefore simpler and potentially cheaper to manufacture than camouflaged logic gates, partially offsetting the larger cell count.

Index Terms—camouflaging, hardware IP protection, universal circuits, reverse engineering, RTL, hardware description language, logic obfuscation, VLSI design

I. INTRODUCTION

In their foundational study of IC camouflaging, Rajendran et al. [5] include a table listing, for each controller in the OpenSPARC T1 processor, the specific algorithmic secret a designer would wish to protect: the decoding algorithm in the instruction decoder, the read/write priority policy in the load-store unit, the division algorithm in the floating-point divider, the interrupt-handling logic in the trap unit, and so on. In other words, the authors *know exactly what the designer wants to hide*—and so, crucially, does the designer herself.

Yet the camouflaging methodology Rajendran et al. propose, and every methodology that has followed it, ignores this knowledge entirely. Gate selection is performed after logic synthesis on a flat gate-level netlist, guided by proxy metrics such as output corruptibility and resolvability that optimise structural properties of the netlist with no formal connection to the algorithmic secrets the designer actually cares about. The designer who wishes to protect her read/write priority logic must herself locate which gates in the synthesised netlist correspond to that logic—a non-trivial, tool-dependent, and

error-prone exercise— and hope that the selected gates, once camouflaged, are sufficient to conceal the algorithm rather than merely some structural artefact of the synthesis tool’s optimisation.

The root cause of this gap is that existing camouflaging flows provide no mechanism for the designer to express her security intent at the level of abstraction at which she works: the register-transfer level (RTL). Physical techniques such as focused ion-beam (FIB) delayering and scanning-electron microscopy (SEM) imaging can recover gate-level netlists from fabricated dies with steadily declining cost [4], making the need for principled, designer-directed camouflaging ever more pressing.

Our proposal. We introduce the `camouflage` HDL construct: a lightweight annotation that a designer places around any VHDL or Verilog statement, block, or part of a statement to declare it as trade-secret logic. The annotation is the designer’s answer to the implicit question posed by Rajendran et al.’s table—it is the mechanism by which she says, directly in her source code, “*this is what I want to protect.*”

A synthesis tool supporting the construct compiles each annotated region into a *universal circuit* [12] (UC): a fixed netlist topology that can realise any Boolean function of the same port signature, configured entirely by a set of constant bits embedded as camouflaged cells during fabrication. El Massad et al. [6] showed that gate-selection-based camouflaging is vulnerable to a satisfiability-based (SAT) oracle attack that recovers hidden functions within minutes; the UC approach sidesteps this by hiding circuit topology entirely, not merely individual gate functions. No designer-specified decoy is required: the UC structure is inherently consistent with every function of the given arity.

This approach gives the designer formal guarantees over exactly what is protected, hides topology as well as function from an adversary who recovers the plain-gate netlist, and produces a predictable $O(s \log s)$ camouflaged cell count at synthesis time.

II. BACKGROUND AND RELATED WORK

A. Camouflaging: attacks and defences

Rajendran et al. [5] formalised IC camouflaging, proposing dummy-contact cells that render an XOR gate physically indistinguishable from a NOR gate. El Massad et al. [6] demonstrated that this and stronger schemes can be broken in minutes by a SAT-based oracle attack on the live chip. Subsequent defences insert cyclic structures [9] or use polymorphic dopant-programmed cells [7] to resist the attack. The present

work addresses *which* logic to camouflage and *how* to compile it, and is orthogonal to the choice of low-level camouflaging cell technology.

B. Logic locking

Logic locking [10] inserts key-gate XOR/XNOR cells so the chip functions correctly only when the correct key is applied at runtime. Unlike camouflaging, it requires secure key distribution and most variants are vulnerable to SAT-based attacks [11]. SAT-resilient schemes such as cyclic locking [9] offer stronger guarantees but at higher overhead or with other trade-offs. Oracle-less attacks [11] target the netlist directly without chip access; our UC-based approach is designed to resist both oracle-guided and oracle-less adversaries, as discussed in Section VI. Our work uses camouflaging exclusively and requires no runtime key.

C. RTL-level obfuscation

ASSURE [1] is the closest prior work to ours in terms of abstraction level: it obfuscates constants, arithmetic operators, and control branches directly in the Verilog RTL, before synthesis. Three differences separate ASSURE from our construct. First, ASSURE is a *logic locking* scheme: the correct behaviour is recovered at runtime by supplying a key stored in tamper-proof memory. Our construct uses *camouflaging*: protection is physical and requires no runtime key or memory. Second, ASSURE targets an untrusted foundry in a netlist-only threat model; our construct targets a reverse-engineer with physical access to the fabricated die. Third, and most fundamentally, ASSURE *automates* the selection of which constructs to obfuscate, guided by a tool-defined policy. Our construct deliberately leaves that decision to the designer. The designer knows which parts of her design constitute trade secrets and which do not—this is precisely the observation that motivates the paper—and she is therefore better placed than any automated tool to make that call.

D. HDL-level IP protection

The closest existing HDL mechanism is `pragma protect` (IEEE Std 1735 [15]), which encrypts a designer-delimited source region so that third-party EDA tools can elaborate but not view it. Two fundamental differences separate `pragma protect` from our construct. First, it protects *source code* from other engineers with EDA tool access; it provides no protection against reverse-engineering of fabricated silicon. Second, because the encrypted text is opaque to the synthesis tool, the tool cannot use it to make security-aware gate selections; our construct is a synthesis directive, not a confidentiality wrapper.

E. Universal circuits

A *universal circuit* UC_s is a Boolean circuit that takes as input both a data vector x and a configuration vector p , and can compute any function $f : \{0, 1\}^k \rightarrow \{0, 1\}^m$ realisable by some circuit of size at most s , by an appropriate choice of p [12]. The p vector simultaneously encodes the function

performed at each programmable gate and the routing of signals through the switching network; it has size $O(s \log s)$. Valiant [12] proved $|UC_s| = O(s \log s)$. Kiss and Schneider [13] gave the first practical implementation; Günther, Kiss, and Schneider [14] subsequently reduced the leading constant to approximately $5s \log_2 s$. Universal circuits have been used in cryptography for private function evaluation [13]; to our knowledge this is the first application to IC camouflaging at the RTL level.

Beerel et al. [2] provide a formal treatment of logic locking security and prove that universal circuits satisfy their proposed security notions; they further show that achieving stronger security than UCs requires cryptographic assumptions. Although their result is stated for logic locking, the security argument applies directly to our camouflaging setting: the UC topology reveals no information about the implemented function. Concurrently and independently, the security of UC-based obfuscation against practical attacks has been studied empirically, with results showing high attack complexity and low key-recovery rates [3]. Shamsi et al. [8] study SAT-attack resistance of UC-based implementations from a related perspective. All three lines of work support the security argument for our approach, which we discuss further in Section VI.

III. THE CAMOUFFAGE CONSTRUCT

A. Design goals

We require the construct to be minimally invasive (marking a region requires no structural change to surrounding HDL), backward-compatible (unaware tools synthesise a correct but unprotected netlist), composable (any number of disjoint regions across multiple files may be marked), decoy-free (the designer need not supply an alternative function), and *region-aware* (the synthesis tool adapts its camouflaging strategy to the nature of the statements inside each marked region).¹

B. Syntax

The construct provides three levels of granularity: a *region* brackets multiple statements, a *statement* is a single annotated statement, and a *sub-expression* brackets part of a line.

Region level (Verilog / SystemVerilog). We adopt the `// synthesis comment-directive` convention, consistent with existing synthesis pragmas such as `translate_off`. Listing 1 shows a region spanning a continuous assignment, a combinational `always` block, a sequential `always` block, and a module instantiation—the kind of multi-statement policy logic that characterises the IP secrets enumerated by Rajendran et al. [5].

```
// synthesis camouflage begin

assign pending = (req & ~(req - 1'b1)); //
combinational
```

¹One might wonder whether adapting the camouflaging strategy to the nature of annotated statements creates an information-theoretic link between the tool output and the region contents. It does not meaningfully do so: the strategy reveals only whether the annotated block is sequential or combinational, a distinction an attacker could determine independently from the unprotected portions of the design.

```

always @(*) begin                                //
    combinational
    grant = 4'b0;
    if (req[0]) grant[0] = 1;
    else if (req[1]) grant[1] = 1;
    else if (req[2]) grant[2] = 1;
    else if (req[3]) grant[3] = 1;
end

always @(posedge clk or posedge rst) begin //
    sequential
    if (rst) starve_cnt <= '0;
    else starve_cnt <= next_starve_cnt;
end

arbiter u_arb (.req(pending), .grant(final_grant))
; // inst.

// synthesis camouflage end

```

Listing 1. A multi-statement camouflage region in Verilog.

Region level (VHDL). We adopt paired comment-pragmas in the style of IEEE 1735’s pragma protect begin/end [15], as shown in Listing 2.

```

-- pragma camouflage begin

pending <= req and (not (req - 1));

arb_proc : process(req) is
begin
    grant <= (others => '0');
    for i in 0 to 3 loop
        if req(i) = '1' then grant(i) <= '1'; exit;
        end if;
    end loop;
end process arb_proc;

seq_proc : process(clk, rst) is
begin
    if rst = '1' then starve_cnt <= (others => '0');
    elsif rising_edge(clk) then starve_cnt <=
        next_starve_cnt;
    end if;
end process seq_proc;

-- pragma camouflage end

```

Listing 2. A camouflage region in VHDL.

Sub-expression level. Because // and -- comments are line-terminated, the region-level directives cannot bracket part of a single line. For sub-expression granularity we introduce camouflage() as a wrapper expression, syntactically identical to a function call and therefore valid anywhere an expression may appear in both languages. Listing 3 shows representative uses.

```

// Verilog: protect only the priority-encoding sub-
// expression;
// the OR with 'base' is synthesised normally.
assign out = base | camouflage(req & ~(req - 1'b1));

// Verilog: protect one branch of a larger
// expression.
assign sched = camouflage(wt[0]*p[0] + wt[1]*p[1])
>> shift;

-- VHDL equivalent
out <= base or camouflage(req and (not (req - 1)));

```

Listing 3. Sub-expression camouflaging via the camouflage() wrapper (Verilog and VHDL).

For backward compatibility, camouflage() may be defined in a vendor library as an identity function; an unaware tool then synthesises the wrapped expression normally, without protection but without error. A supporting tool extracts the wrapped expression, treats it as a self-contained combinational function of its free variables, and compiles it to a UC inlined at that point in the netlist. Optionally, the tool may also camouflage which signals the expression depends on by interposing a camouflaged permutation on the UC’s inputs—the same mechanism applies to any subregion type, including module-instantiation port mappings.

C. Semantics

A protected region R may contain statements of three distinct kinds, each of which exposes a different notion of “the function being hidden” and therefore requires a different camouflaging strategy. The synthesis tool classifies each statement in R and applies the corresponding treatment.

Combinational subregions. Continuous assignments and combinational always blocks (always @(*)) jointly define a combinational function $f_{\text{comb}} : \{0, 1\}^k \rightarrow \{0, 1\}^m$ from their collective input signals to their collective output signals. The tool compiles f_{comb} into a universal circuit UC_s , finding p^* such that $UC_s(x, p^*) = f_{\text{comb}}(x)$ for all x . Each bit of p^* is embedded as a camouflaged tie cell. An adversary who recovers the UC topology learns only the port widths (k, m) and an upper bound s on the combinational complexity of f_{comb} .

Sequential subregions. A sequential always block implements a finite state machine with two separable components: the *next-state and output logic* $\delta(Q, x)$, and the *activation signals*—which input is the clock edge, which is reset, and what value is loaded on reset. The tool treats these separately. The next-state and output logic δ is compiled to a UC exactly as in the combinational case. The activation signals are camouflaged by inserting a camouflaged multiplexer (a two-input MUX whose select line is driven by a camouflaged tie cell) before each flip-flop’s clock and reset inputs: the multiplexer selects among a small set of n candidate signals. To the reverse-engineer, it is ambiguous which candidate is the true clock and which is the true reset, as well as what state is loaded on reset; the attacker must consider $n^2 \cdot 2^w$ combinations, where w is the width of the reset state. Note that inserting logic on the clock path is timing-sensitive: the additional MUX introduces a small but non-zero delay that must be accounted for in static timing analysis. In practice, a designer wishing to avoid clock-path manipulation may restrict activation-signal camouflaging to the reset input only, at the cost of a reduced ambiguity set.

Sub-expression subregions. The camouflage() wrapper brackets a single expression e within an otherwise unprotected statement. The tool extracts the combinational function $f_e : \{0, 1\}^k \rightarrow \{0, 1\}^m$ computed by e , compiles it to

a UC instantiated inline, and synthesises the remainder of the enclosing statement normally. This enables finer-grained protection and consequently finer control over camouflaging overhead. In particular, many IP secrets are encoded not in the logic structure of an `always` block but in the *constants* within it: a priority threshold, a scheduling weight, a timeout value. Protecting such a secret may require nothing more than wrapping the relevant constant in `camouflage()`, leaving the surrounding control logic fully visible and incurring UC overhead only on the secret value itself.

Module-instantiation subregions. If the instantiated module is defined in the same design (i.e., not a black-box IP), the tool recursively treats its body as a protected region, and may additionally camouflage the port-to-signal mapping via a camouflaged permutation network as described above.

IV. SYNTHESIS METHODOLOGY

A. Marking and port inference

After parsing, every intermediate representation (IR) node derived from a tagged statement receives a `PROTECT` flag. Standard optimisation passes may proceed within the flagged cone, but a flagged signal may not be propagated as a constant into an unflagged node. The tool infers the port signature (k, m) : k is the number of signals crossing into the flagged cone from outside, comprising primary inputs that drive at least one flagged node and outputs of unflagged nodes that drive at least one flagged node; m is the number of flagged nodes that drive unflagged or primary-output nodes.

B. UC instantiation and programming

Given (k, m) and the post-optimisation flagged-cone size s , the tool selects a UC of capacity s from a pre-characterised library based on the Günther et al. construction [14], instantiating $\approx 5s \log_2 s$ plain gates. It then runs a UC programming algorithm to derive p^* .

C. Configuration bit embedding

Each bit p_i^* is instantiated as a camouflaged tie cell. Such cells are already offered by several foundry PDKs and are substantially cheaper to manufacture than camouflaged logic gates, since they must hide only a single bit rather than a gate function.

V. CASE STUDY: OPENSPARC T1

The OpenSPARC T1 is an open-source eight-core SPARC processor [16]. We protect two components whose scheduling algorithms constitute plausible trade secrets: the **thread scheduler** in `sparc_ifu_sw1`, which determines which of four hardware threads receives an issue slot each cycle, and the **memory-request arbiter** in `l2_dir_dp`, which prioritises L2 cache fill, snoop, and CPU-side requests. These two modules were selected because they appear in Rajendran et al.’s [5] enumeration of IP secrets (Table 16(a) of that paper), making the comparison direct.

Annotation effort. Protecting each component requires a `// synthesis camouflage begin/end region`

TABLE I
ESTIMATED OVERHEAD FOR UC-BASED CAMOUFLAGING ON TWO OPENSPARC T1 COMPONENTS (s = PLAIN GATE COUNT OF PROTECTED CONE). “BLOCK OVHD.” IS RELATIVE TO THE PROTECTED BLOCK’S OWN GATE COUNT; “CHIP OVHD.” IS RELATIVE TO THE FULL T1 GATE COUNT ($\approx 34\text{M}$ GATES).

Component	s	UC gates	Camo. cells	Block ovhd.	Chip ovhd.
Thread scheduler	120	$\approx 3,900$	≈ 840	$\approx 32\times$	$\approx 0.18\%$
Memory arbiter	85	$\approx 2,700$	≈ 595	$\approx 32\times$	$\approx 0.13\%$
Total	205	$\approx 6,600$	$\approx 1,435$	$\approx 32\times$	$\approx 0.31\%$

around one `always` block. No decoy, no gate budget, and no post-synthesis inspection are required.

Overhead analysis. Table I summarises the estimated UC size and camouflaged cell count, using the Günther et al. construction ($5s \log_2 s$ gates). Gate counts for the plain logic are estimated from the publicly available OpenSPARC synthesis reports. The $\approx 32\times$ block-level overhead reflects the $O(s \log s)$ UC expansion; relative to the full T1 chip it is negligible (0.31%). Timing impact is not estimated here, as it requires an actual synthesis run; however, since the UC consists of plain combinational cells, its critical path can be analysed by standard static timing analysis and is bounded by the depth of the Günther et al. graph construction, which is $O(\log^2 s)$.

Security comparison. Under per-gate camouflaging, the topology of the protected logic—wires, fanout, cone structure—remains visible and is exploited by SAT-based attacks [6]. Under the UC approach, topology is entirely hidden inside the UC structure. An attacker who recovers every plain gate on the chip still has no information about the protected function beyond its port widths. Furthermore, the 1,435 camouflaged tie cells hide single bits rather than gate functions, and are likely cheaper per cell than camouflaged logic gates, partially offsetting the larger plain-gate UC area.

VI. DISCUSSION

Security guarantees. The UC approach guarantees that physical inspection of the layout reveals nothing about the protected function beyond its port widths: the UC topology is identical regardless of which function it implements. A SAT-oracle attack [6], if successful, would recover the function from black-box input-output access to the live chip combined with knowledge of the unprotected logic. Should that occur, it is not a failure of the camouflaging: it means the protected function was learnable from observable chip behaviour, a property of the function itself against which no layout-level protection can defend. Shamsi et al. [8] have studied SAT-attack resistance of UC-based implementations in this light. A separate and subtler question is whether the unprotected portions of the chip leak information about the protected function through their outputs without full oracle access; we leave a formal treatment of this channel to future work.

Area trade-off. The $O(s \log s)$ plain-gate overhead exceeds the size of the original protected circuit, and the total cell count is larger than in the per-gate approach. The practical

trade-off depends on foundry pricing for camouflaged logic gates versus camouflaged tie cells: if tie cells are significantly cheaper, the UC approach may reduce total cost despite the larger plain-gate footprint. When the secret is encoded primarily in constants—as argued above for priority and weight parameters—the relevant s is the number of constant bits rather than the full block gate count, making the overhead negligible.

Compositionality. Multiple annotated regions produce independent UCs. If an output of a UC drives observable unprotected logic, that logic may reveal partial information about the protected function; whether to annotate additional logic or accept this leakage is left to the designer’s judgement.

Limitations and future work. This paper is a design proposal. Validation requires extending Yosys [18] with UC instantiation and tie-cell embedding passes, and characterising camouflaged tie cells in a standard PDK. Camouflaging activation signals may be partly defeated if a reverse-engineer identifies the clock from the physical clock-distribution network independently; whether this falls within the threat model is a design-point decision. A promising direction is to protect marked regions such that any syntactically valid HDL of the same length is plausible—a code-level analogue of Holcomb et al.’s notion of circuits with multiple viable functions [17]—as if the designer had drawn a shade over the annotated source. Finally, AI-assisted analysis could warn the designer when a protected region would be trivially revealed by observable unprotected logic, closing the leakage channel noted in the security discussion.

VII. CONCLUSION

We have proposed the `camouflage` HDL construct, which lets a designer declare trade-secret logic at the RTL level by annotating statements in VHDL or Verilog. A synthesis tool supporting the construct compiles each annotated region into a universal circuit whose topology is publicly visible but function-agnostic, embedding the configuration bits that encode the protected function as camouflaged constant cells. The approach provides a stronger hiding guarantee than per-gate camouflaging—circuit topology is hidden, not merely individual gate functions—eliminates the need for designer-specified decoys, and concentrates camouflaging overhead in simple tie cells rather than complex logic gates. Applied to the thread scheduler and memory arbiter of the OpenSPARC T1 processor, the scheme requires annotating two `always` blocks and yields approximately 1,435 camouflaged cells at an estimated 0.31% chip area overhead. We plan to validate the approach by extending Yosys with the required synthesis passes and evaluating SAT-attack resistance of the resulting netlists.

REFERENCES

- [1] C. Pilato, A. Chowdhury, D. Sciancalepore, N. Chandrachodan, and S. Garg, “ASSURE: RTL locking against an untrusted foundry,” *IEEE Trans. VLSI Syst.*, vol. 29, no. 7, pp. 1306–1318, 2021.
- [2] P. Beerel, M. Georgiou, B. Hamlin, A. J. Malozemoff, and P. Nuzzo, “Towards a formal treatment of logic locking,” *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2022, no. 2, pp. 92–114, 2022.
- [3] Anonymous, “Security analysis of universal circuits as a mechanism for hardware obfuscation,” arXiv preprint arXiv:2604.03396, 2025.
- [4] R. Torrance and D. James, “The state-of-the-art in IC reverse engineering,” in *Proc. CHES*, 2009, pp. 363–381.
- [5] J. Rajendran, M. Sam, O. Sinanoglu, and R. Karri, “Security analysis of integrated circuit camouflaging,” in *Proc. ACM CCS*, 2013, pp. 709–720.
- [6] M. El Massad, S. Garg, and M. V. Tripunitara, “Integrated circuit (IC) decamouflaging: Reverse engineering camouflaged ICs within minutes,” in *Proc. NDSS*, 2015.
- [7] V. Iyengar, S. Bhanja, and R. Karri, “MSOFT: Metamorphic standard-cell obfuscation for protecting intellectual property,” *IEEE Trans. VLSI Syst.*, vol. 26, no. 8, pp. 1533–1545, 2018.
- [8] K. Shamsi, D. Z. Pan, and Y. Jin, “On the impossibility of approximation-resilient circuit locking,” in *Proc. IEEE HOST*, 2019, pp. 161–170.
- [9] K. Shamsi, M. Li, T. Meade, Z. Zhao, D. Pan, and Y. Jin, “Cyclic obfuscation for creating SAT-unresolvable circuits,” in *Proc. ACM/IEEE GLSVLSI*, 2017, pp. 173–178.
- [10] J. Rajendran, H. Zhang, C. Zhang, G. S. Rose, Y. Pino, O. Sinanoglu, and R. Karri, “Fault analysis-based logic encryption,” *IEEE Trans. Comput.*, vol. 64, no. 2, pp. 410–424, 2015.
- [11] P. Subramanyan, S. Ray, and S. Malik, “Evaluating the security of logic encryption algorithms,” in *Proc. HOST*, 2015, pp. 137–143.
- [12] L. G. Valiant, “Universal circuits (preliminary report),” in *Proc. ACM STOC*, 1976, pp. 196–203.
- [13] Á. Kiss and T. Schneider, “Valiant’s universal circuit is practical,” in *Proc. EUROCRYPT*, 2016, pp. 699–728.
- [14] D. Günther, Á. Kiss, and T. Schneider, “More efficient universal circuit constructions,” in *Proc. ASIACRYPT*, 2017, pp. 443–470.
- [15] IEEE Std 1735-2014, “IEEE Recommended Practice for Encryption and Management of Electronic Design Intellectual Property (IP),” 2014.
- [16] Sun Microsystems, “OpenSPARC T1 Microarchitecture Specification,” 2006, <http://www.opensparc.net>.
- [17] S. Keshavarz, C. Paar, and D. E. Holcomb, “Design automation for obfuscated circuits with multiple viable functions,” in *Proc. DATE*, 2017, pp. 886–889.
- [18] C. Wolf, “Yosys Open SYnthesis Suite,” <http://www.clifford.at/yosys/>, 2013.